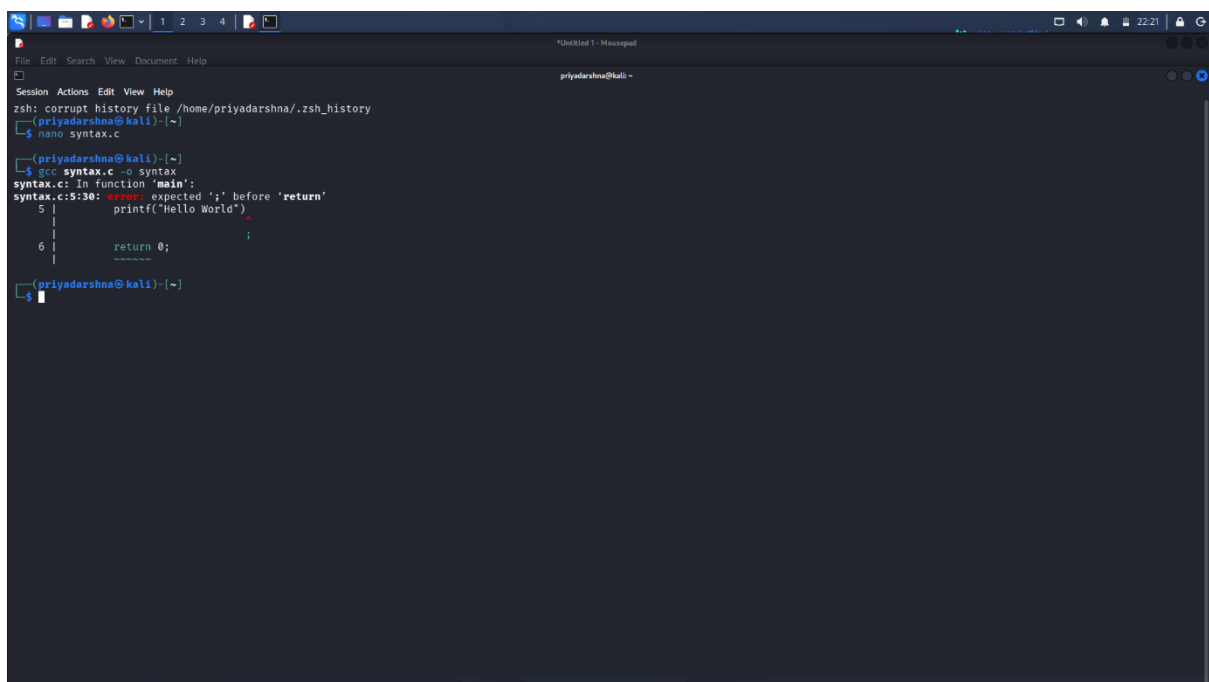


Experiment – 3

Topic – C programming -Debugging Tools

1. Error Classification: Write a small programming snippet illustrating each of the following error types:

a) Syntax Error:

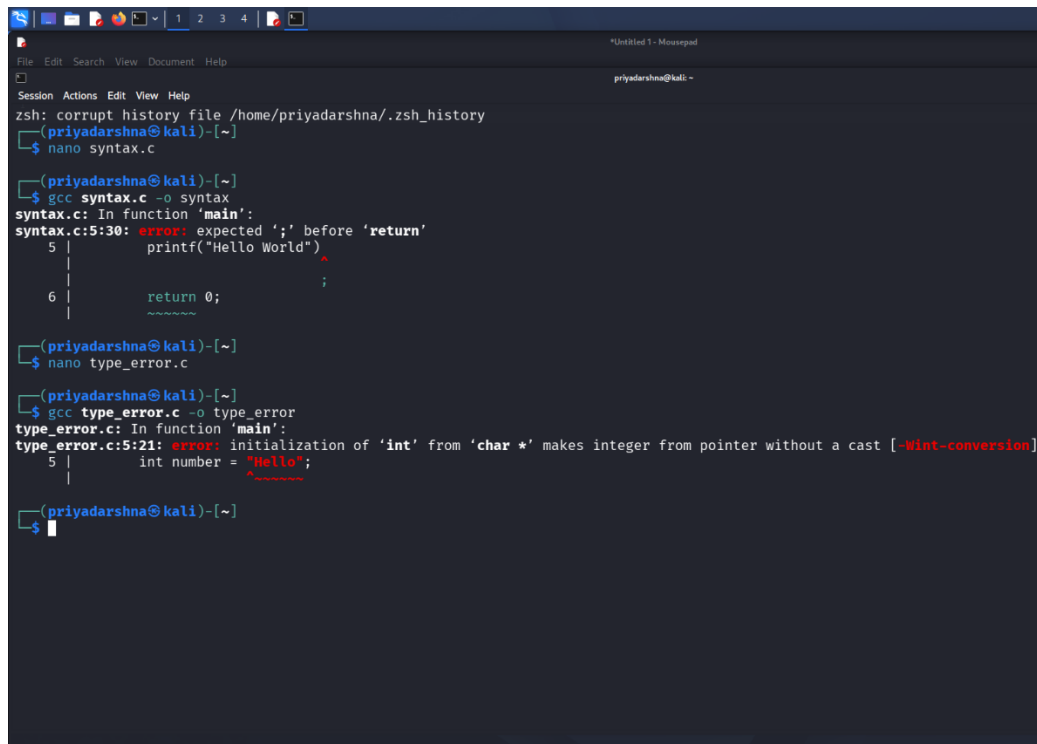


The screenshot shows a terminal window with the following content:

```
zsh: corrupt history file /home/priyadarshna/.zsh_history
(priyadarshna@kali)~$ nano syntax.c
(priyadarshna@kali)~$ gcc syntax.c -o syntax
syntax.c: In function 'main':
syntax.c:5:30: error: expected ';' before 'return'
5 |         printf("Hello World")
  |                               ^
6 |         return 0;
  |         ~~~~~~
(priyadarshna@kali)~$
```

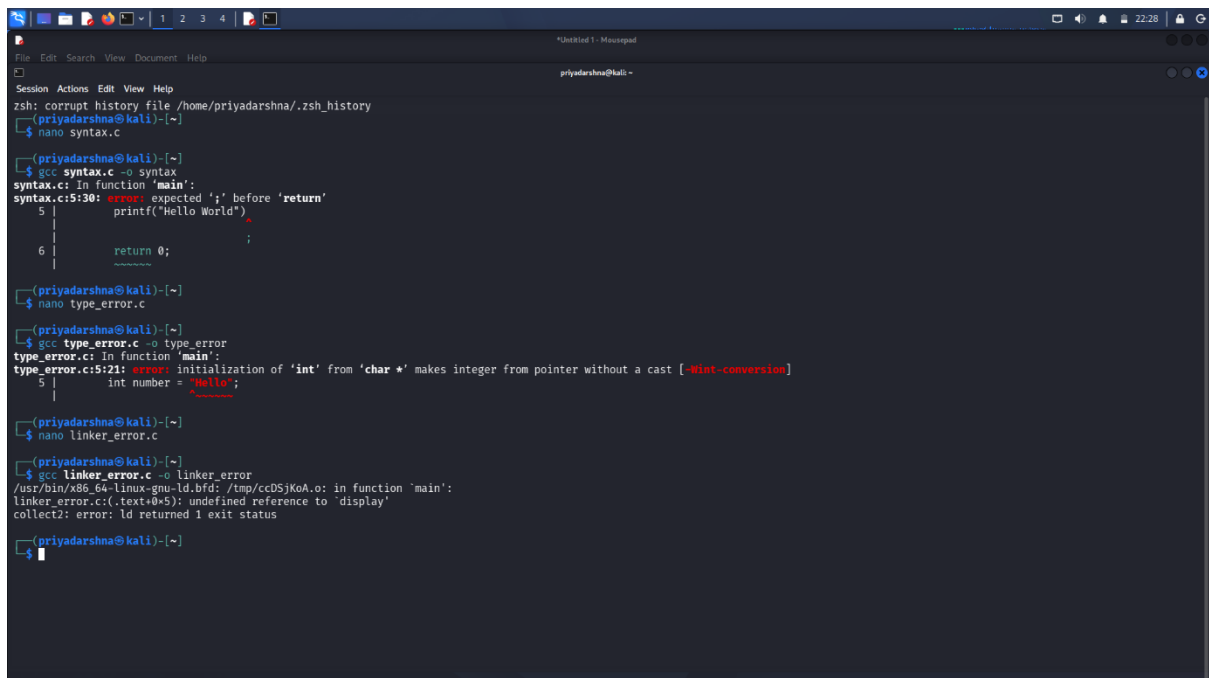
The error message indicates a syntax error in the C program. The error is located at line 5, column 30, where the compiler expected a semicolon before the 'return' statement. The code snippet shows a 'printf' statement followed by a 'return' statement without a semicolon.

b) Compilation/Type error:



```
zsh: corrupt history file /home/priyadarshna/.zsh_history
(priyadarshna@kali)~$ nano syntax.c
(priyadarshna@kali)~$ gcc syntax.c -o syntax
syntax.c: In function 'main':
syntax.c:5:30: error: expected ';' before 'return'
   5 |         printf("Hello World")
     |                             ^
   6 |         return 0;
     |         ~~~~~
(priyadarshna@kali)~$ nano type_error.c
(priyadarshna@kali)~$ gcc type_error.c -o type_error
type_error.c: In function 'main':
type_error.c:5:21: error: initialization of 'int' from 'char *' makes integer from pointer without a cast [-Wint-conversion]
   5 |         int number = "Hello";
     |                     ^~~~~~
(priyadarshna@kali)~$
```

c). Linker error:



```
zsh: corrupt history file /home/priyadarshna/.zsh_history
(priyadarshna@kali)~$ nano syntax.c
(priyadarshna@kali)~$ gcc syntax.c -o syntax
syntax.c: In function 'main':
syntax.c:5:30: error: expected ';' before 'return'
   5 |         printf("Hello World")
     |                             ^
   6 |         return 0;
     |         ~~~~~
(priyadarshna@kali)~$ nano type_error.c
(priyadarshna@kali)~$ gcc type_error.c -o type_error
type_error.c: In function 'main':
type_error.c:5:21: error: initialization of 'int' from 'char *' makes integer from pointer without a cast [-Wint-conversion]
   5 |         int number = "Hello";
     |                     ^~~~~~
(priyadarshna@kali)~$ nano linker_error.c
(priyadarshna@kali)~$ gcc linker_error.c -o linker_error
/usr/bin/ld: /tmp/ccDSjKoA.o: in function 'main':
linker_error.c:(.text+0x5): undefined reference to 'display'
collect2: error: ld returned 1 exit status
(priyadarshna@kali)~$
```

d). Runtime error:

```
Session Actions Edit View Help
zsh: corrupt history file /home/priyadarshna/.zsh_history
(priyadarshna@kali)~$ gcc -g runtime.c -o runtime
(priyadarshna@kali)~$ gdb ./runtime
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./runtime...
(gdb) run
Starting program: /home/priyadarshna/runtime
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
0x000055555555514d in main () at runtime.c:6
6      p=10;
(gdb) print p
$1 = (int *) 0x0
(gdb) print
$2 = (int *) 0x0
(gdb) print *p
X Cannot access memory at address 0x0
(gdb) 
```

e). Logical error:

```
Session Actions Edit View Help
zsh: corrupt history file /home/priyadarshna/.zsh_history
(priyadarshna@kali)~$ nano logical.c
(priyadarshna@kali)~$ gcc -g logical.c -o logical
(priyadarshna@kali)~$ gdb ./logical
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./logical...
(gdb) break main
Breakpoint 1 at 0x1141: file logical.c, line 5.
(gdb) run
Starting program: /home/priyadarshna/logical
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at logical.c:5
5      int a=10,b=5;
(gdb) next
6      int result=a-b; //logical error:should be additionn
(gdb) next
8      printf("Result:%d\n",result);
(gdb) print a
$1 = 10
(gdb) print b
$2 = 5
(gdb) print result
$3 = 5
(gdb) 
```

2. GNU Concept: Explain what the GNU project is and how GDB fits into the GNU ecosystem.

The GNU Project is an initiative to create a completely free, Unix-like operating system, while GDB is its core debugging tool.

THE GNU PROJECT-

*PURPOSE: Launched in 1983 by Richard Stallman to give users total freedom to give users total freedom to run, share, study, and modify software.

*LEGACY: It develops the essential software suite (compiler, text, editor, shell) that forms the backbone of modern linux systems (correctly called GNU/Linux)

How GDB Fits in:

*Role: GDB (GNU Debugger) is the official, standard debugging tool for the GNU ecosystem.

*Function: It allows developers to inspect what is happening inside a program while it executes or when it crashes.

*Importance: It provides the critical debugging infrastructure needed to build, test and maintain robust free software within the system.

3. Shell Context: Define what a shell and kernel are. Briefly list three common Linux shells (e.g., bash, zsh, fish).

*kernel: Core operating system layer interacting directly with physical hardware.

*Shell: Command-line interpreter that reads user inputs and executes programs.

Three Common Linux Shells:

*Bash: Default, standard shell on most linux systems.

*Zsh: Advanced shell offering highly customizable themes and autocompletion.

*Fish: User-friendly shell focusing on out-of-the-box syntax highlighting.

4. Research and contrast the difference between:
Breakpoint, Watchpoint, Catchpoint

Feature	Breakpoint	Watchpoint	Catchpoint
Tigger condition	Reaching a specific line of code or function.	When a variable or memory address is modified or read.	When a specific system event or language exception occurs.
Primary Scope	Code/Instruction execution.	Data/Memory states.	Program events/Exceptions.
Common use case	Stopping at the start of a function to step through code.	Finding out exactly where a variable is getting corrupted.	Stopping execution right when an unhandled error is thrown.

Name: Priyadarshna

Batch: 15

SAPID: 590032928

Topic: Debugging

